Mandatory Exercise TEK5600

Visualization of vector fields

In this exercise, you will visualize two vector fields using two different methods: (1) a geometry-based approach using field lines, and (2) a texture-based approach using Line Integral Convolution (LIC).

In Exercise 1, you will visualize vector fields using field lines, while in Exercise 2, you will apply LIC.

The assignment consists of implementing both techniques, comparing the results, and summarizing your findings in a final report.

Exercise 1

In the first exercise, you are going to do the following tasks:

1. Write a program that reads two vector data sets stored in the HDF5 data format.

2. Make a field line integrator based on both forward Euler and on the 4'th order Runge Kutta integration method. (You will need this for both task 1 and 2). Figure 1 shows a simple field line for one of the data sets.

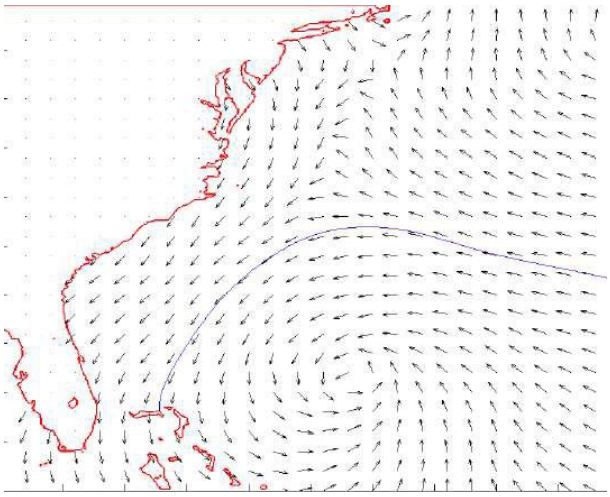

Figure 1: Isabel data visualized by a single field line and arrows.

3. Visualize the two vector fields using field lines. Try out various seed point strategies. Seed points are the starting position or starting point for the field lines to be computed. The visualization result depends strongly on the selection of seed points. Examples of different seeding strategies could be *random based*, *uniform based*, *density based* and *flow feature based*.

   <u>You should implement at least three different seeding strategies</u> and compare their results. However, you are encouraged to experiment with additional strategies. Additionally, examine how changing the number of seed points influences the behavior of each seeding strategy.

In addition, try out the effect of varying the lengths of the field lines and the effect of changing the accuracy of the integration scheme (first order Euler or fourth order Runge Kutta). The accuracy of the computed field lines will depend on both the integration scheme used as well as the chosen step size. *Hint: The biggest change in result from using integration methods of different accuracy will occur in regions of the flow with high curvature.*

4. **Optional** (but encouraged)

   Use the results from your tests with the different seeding strategies, and combine them with your visualization knowledge to create the best possible field-line visualization for both data sets. Consider, for example, how choices of color, opacity, and your most effective seeding strategy can enhance the final visualization.

5. Write a report including the following:
   a. Short introduction including the definition of a field line as well as the description of the integration methods.
   b. Evaluate the results of applying different integration methods (Euler and Runge Kutta) to the two datasets.
   c. Describe your selection of seeding strategies with sufficient details.
   d. Compare the selection of seeding strategies and comment on the resulting visualization results.
   e. Comment on the differences and which method do you think gives the better result?
   f. (**optional**): Create your best visualization of the two data sets using what you learned (so far) in the course.

Exercise 2

In the second exercise, you are going to visualize the same two vector fields using Line Integration Convolution (LIC). Some of the work done in exercise 1 will be useful for this task. Do the following steps:

1. Introduction - Write a short introduction about Line Integral Convolution by describing the LIC algorithm. Do present the most relevant formulas.

2. Tell short about your LIC implementation.

3. Compare the results obtained by Euler and Runge Kutta integration methods. *(Any thoughts about which method is more sensitive to the accuracy of the integration method – geometry-based or texture-based methods?)*

4. Experiment with different filter-kernel lengths for the LIC and compare the results. How do the changes affect both the visualization outcome and the computational time?

5. **Optional 1**: Try using a higher resolution on the input and output texture rather than just using the same resolution on the textures as is defined by the vector field. *(Line integral Convolution is independent of the resolution of the vector field. As an example,*

the right image of Figure 2 is created by using a higher resolution of the texture compared to the associated vector field).

6. **Optional 2**: Use the knowledge you have gained in the course to create the best possible visualization using Line Integral Convolution.

7. Compare the results from Exercises 1 and 2—that is, compare the geometric field-line visualizations with the LIC-based textures. Describe the advantages and limitations of each class of methods.

8. Write a short conclusion/summary.

Some information about the data

The two datasets to be used in the exercise is taken from weather simulations. They can be downloaded from the homepage of the course.

The first data set *"Metsim"* is generated by the weather forecast model "Weather Research and Forecasting model (WRF)" developed by the university of Washington, while the second data set *"Isabel"* is a simulation of a tropical hurricane done by "National Center for Athmospheric Research".

Both data sets are stored in the data format HDF5, with the same internal hierarchical representation:
Velocity (group name)

- X-comp (data array name)
- Y-comp (data array name)

Here, the velocity field is given by V(x,y) = [X-comp(x,y),Y-comp(x,y)]. The resolution of the Metsim data is 127 points in both x- and y- direction. The resolution of the Isabel data is 500 points in both x- and y- direction.

NOTE!!!!!
Both datasets are stored as two-dimensional arrays, one array for each component. However, there is a significant difference in how the data sets are created. For Metsim, the x-index is the first index, while for the Isabel data set the y-index is the first index. *The reason for this is that different program languages may have different ways of storing the data arrays (or matrices) internally.*

You can put the data in a local two-dimensional array as follows:

Isabel:

```
for (int i = 0; i < dim[0]; i++)
 for (int j = 0; j < dim[1]; j++) {
   I[i][j] = dataarray[ j + (i * dim[1])];
 }
```

metsim:

```
for (int j = 0; j < dim[1]; j++)
 for (int i = 0; i < dim[0]; i++) {
   I[i][j] = dataarray[ i + (j *dim[0])];
 }
```

Figure 2 shows examples of LIC pictures of the two vector fields. These pictures can be used to check whether you have done the coding right.

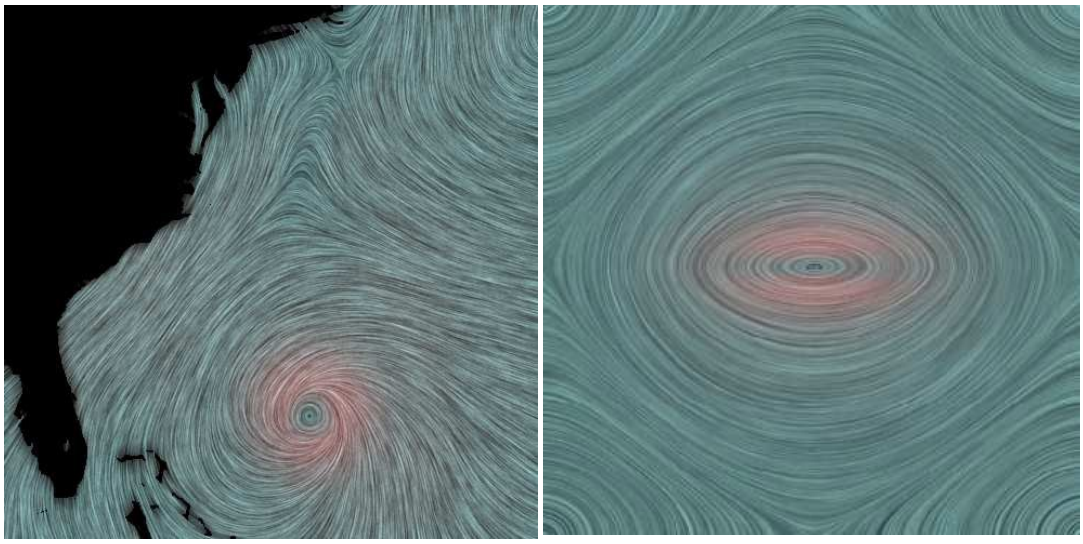

Figure 2: LIC-images of Isabel (right) and metsim (left).